

Architecture

Overview

```
Browser (Next.js) --HTTP/JSON--> Express API (port 4000) --pg--> PostgreSQL
/admin, /audit                      routes + hashChain + rbac      users, resources,
                                                               permissions, ledger_events
```

Three tiers: a Next.js client, a stateless Express API, and PostgreSQL. All tamper-evidence logic lives in the API layer (`hashChain.js`); the database stores plain rows.

Data model

Table	Purpose	Columns
users	People and roles	id, name, role
resources	Things access is granted to	id, name
permissions	Current access state (read by the admin dashboard)	id, user_id, resource_id, access_type, granted_by, created_at; unique on (user_id, resource_id, access_type)
ledger_events	History , the hash chain	id, prev_hash, event_hash, actor_id, resource_id, action, created_at

`permissions` can be changed freely. `ledger_events` is the evidence trail. `ledger_events.created_at` is `TEXT` (an ISO-8601 string) on purpose: the exact same string must be hashed on insert and on re-verify, and a native `TIMESTAMP` can round-trip differently through `pg`.

Hash chain

```
event_hash = SHA256(prev_hash + actor_id + resource_id + action + created_at)
```

Fields are concatenated as strings with no delimiter. The first event's `prev_hash` is `"0".repeat(64)`. `action` is `grant:<access_type>` (for example `grant:read`) or `revoke`.

Writing (`insertLedgerEvent`): reads the latest `event_hash`, computes the new hash with the current ISO timestamp, and inserts. Calls are chained through a promise queue so two simultaneous requests cannot read the same "latest hash" and fork the chain.

Verifying (`verifyChain`): walks all rows in `id` order, recomputing each hash from the previous row's stored hash. A row is added to `brokenAt` if its stored `prev_hash` or its `event_hash` doesn't match what was expected. It returns `{ valid, brokenAt: [ids] }`.

Request flows

Grant (`POST /access/grant`)

1. Validate required fields.
2. `canAccess(granted_by, resource_id, 'admin')`; reject with 403 if not admin.
3. Insert into `permissions` (duplicate grants fail on the unique constraint).
4. `insertLedgerEvent` with action `grant:<access_type>`.
5. Return `{ permission, ledgerEvent }`.

Revoke (`POST /access/revoke`): same admin check, then deletes all `permissions` rows for that user and resource, then appends a `revoke` ledger event. Returns 404 if nothing matched.

Audit (`GET /audit/verify` + `GET /ledger`): the audit page fetches both, marks every ID in `brokenAt` as tampered,

and shows the newest flagged event most prominently.

Access control

`canAccess(pool, userId, resourceId, accessType)` : unknown users get no access; users with role `admin` always pass; everyone else needs a matching row in `permissions` . Grant and revoke require the acting user to pass an `admin` check.

Client structure

- `pages/index.js` : landing page
- `pages/admin.js` : login, access matrix, grant/revoke composer, receipts
- `pages/audit.js` : verification status card and ledger events table
- `lib/session.js` : session in `localStorage`
- `lib/adminApi.js` , `lib/api.js` : fetch wrappers
- `lib/directory.js` : fallback user/resource names, because the API has no `/users` or `/resources` route yet

Deployment

- Database: Supabase PostgreSQL (SSL enabled by `pool.js` for any non-localhost URL)
- API: any Node host; reads `DATABASE_URL` and `PORT` from environment
- Client: any Next.js host; set `NEXT_PUBLIC_API_BASE` to the API URL